Disentangling Localization from Editing in Instruction-Guided Image Manipulation

Amritha S (23BEC1368) Yugeshwaran P (23BEC1404)
School of Electronics Engineering, VIT Chennai
Guide: Dr. Saranya M (54783)
Live system: https://localize-edit.onrender.com

***Abstract*—When you ask an image editor to "change the cat to a fox", you tell it what to change but not where. In our tests an off-the-shelf inpainting editor that was given no region changed 93.1% of the pixels it should have left alone. We therefore pulled the question of *where* out of the editor and treated it as its own prediction task, scored against the pixels the edit really changed. Our first model is a small head of 4.69 M parameters that we trained from scratch on frozen FastSAM and CLIP features. On 503 held-out MagicBrush edits it reaches an IoU of 0.2065, which is higher than CLIPSeg (150.7 M parameters, 0.1849) and higher than the masks people drew for the dataset (0.1511). When we benchmarked newer open models, Grounding DINO with SAM 2.1 did better still (0.2721), so we built our final model on top of it; with two-fold cross-validation it scores 0.2771. Pixels outside the region are copied back from the input after every edit. The whole system runs as a public website with live GPU inference, and after two follow-up prompts the base paper's mask mechanism had altered 73.6% of the photo against 21.1% for ours.**

***Index Terms*—image editing, edit localization, referring segmentation, diffusion inpainting, MagicBrush**

I. Introduction

A. Problem

Current editors such as Step1X-Edit [11], FLUX.1 Kontext [12], Qwen-Image [13] and GPT-4o [14] take a photo and a sentence and give back a new photo. The sentence tells the model what to change. Which pixels it is allowed to touch is left for the model to work out on its own, and the user never gets to see that decision. If the model guesses badly, things that should have stayed the same get redrawn too, and the user only finds out after looking closely at the result. The GPT-ImgEval study [14] found this with GPT-4o, whose brush tool still regenerates the entire image, and it found that both GPT-4o and Gemini 2.0 Flash drift further from the original as more edits are chained.

People usually work around this by drawing a mask by hand. It helps. But now a person is doing the localization, and even the dataset masks are generous. In MagicBrush a drawn mask covers 62.4% of the image on average, while the pixels that actually change cover only 10.3%.

TABLE I
THE 45 RECENT WORKS, GROUPED BY HOW THEY TREAT THE EDIT REGION.

Treatment of the region	Works	Examples
Takes a region as input	17	[4], [9], [10], [19]
Reads one from a running generator	11	[2], [3], [5], [8]
Scores or surveys the task	15	[6], [7], [15]
Predicts it from the instruction	**2**	[1], [18]

B. Objective

We want to predict the edit region from just the photo and the instruction, cheaply enough that it can sit in front of any editor, and we want to check how close that region is to the area the edit really covers. Our base paper [1] also predicts a region, but it never says how accurate that region is.

C. What we did

We trained a small head (4,693,633 parameters) from scratch on frozen features and compared it, at the same size, with the base paper's mask equation. Its training targets come from the colour difference between each before and after photo (CIE76 ΔE) rather than from hand-drawn masks. We then benchmarked a few recent open localizers on the same test split, built a fused model (ours v2) on the best one, and put the whole thing on a website that checks after every run that no pixel outside the region was changed.

II. Literature Survey

A. Survey method

We went through 50 papers, 45 of them published in 2024 or later. Grouping them by year did not tell us much. Instead we grouped them by what each one does with the edit region (Table I). The papers closest to our work are summarised in Table II, and each title there is a link to the paper.

B. What the survey shows

Most papers either assume someone has already supplied the region, or pull it out of a generator while it is running. Benchmarks like LocateEdit-Bench look

TABLE II
LITERATURE SURVEY. TITLES ARE LINKS TO THE PAPERS.

Work	Year	Objective and approach	Limitation for our task
Edit Where You Mean (AdaptEdit) [1] **[base]**	2026	Mask-free local editing; adapters in a frozen 20.43 B DiT, a SpatialGate and a 7.8 M MaskPredictor	Mask accuracy never reported; trained on 8×80 GB GPUs; code not released
Rethinking Where to Edit [2]	2026	Training-free localization from attention of source and target streams	Unsupervised; keeps one component, so repeated objects break
WhereEdit [3]	2026	One-step editing with an AutoMask built from cross-attention	Mask exists only inside its own generator
MaskFlow [4]	2026	Puts the mask into the flow-matching path for clean regional edits	Needs the mask as input
EditMGT [5]	2026	Masked generative transformer; sharper attention-based localization	Region read from attention, not trained or scored
LocateEdit-Bench [6]	2026	231K edited images to benchmark where an edit happened	Scores edits after they occur, not a region predicted before
RegionE [8]	2025	Skips denoising in unchanged regions for speed	Region inferred after generation starts
SpotEdit [9]	2025	Selective regional editing in diffusion transformers	Consumes a region, does not produce one
Step1X-Edit [11]	2025	MLLM plus DiT general editor; introduces GEdit-Bench	Region implicit, never exposed
FLUX.1 Kontext [12]	2025	Flow-matching editor with multi-turn consistency	12 B; region implicit
Qwen-Image [13]	2025	20 B MMDiT generator and editor with dual encoding	Region implicit; too large for a laptop
GPT-ImgEval [14]	2025	Benchmarks GPT-4o editing; compares with Gemini 2.0 Flash	Finds whole-image regeneration and drift over turns
Region in Context [16]	2025	Reasons about a region and its surroundings	Region tied to existing content
RePlan [17]	2025	MLLM plans regions for complex instructions	Heavy MLLM in the loop
MADiff [18]	2024	Predicts a mask, then edits; fashion images	One domain; needs a position prompt
LazyDiffusion [19]	2024	Generates only masked pixels for interactive editing	Needs a user-drawn mask
SAM 2 [20]	2024	Promptable segmentation for images and video	Needs a point or box prompt
Grounding DINO [21]	2023	Open-set detection from a text phrase	Boxes only; finds what is named, not what changes
FastSAM [22]	2023	YOLOv8-seg with 32 prototypes; fast segment-anything	Class-agnostic; no language input
MagicBrush [23]	2023	10K manually annotated multi-turn edit triplets	Drawn masks are loose (62.4% of frame)
CLIPSeg [25]	2021	Text-prompted segmentation on CLIP	Referring semantics, weak on insertion

at where an edit ended up once it is finished. We found only two papers that predict the region from the instruction itself. One of them, MADiff, only handles clothing, so the base paper is the one that matches our setting.

C. Base paper and the gap

AdaptEdit [1], published at NeurIPS 2026, adds adapters to all 60 blocks of a frozen 20.43 B diffusion transformer, along with a 7.8 M MaskPredictor so that the user does not need to draw a mask. The paper reports image-quality scores on three benchmarks, but none for the mask. The closest it comes is saying the mask behaves roughly like a ground-truth mask shrunk by 16 pixels. Training its 5,033 M parameters took eight 80 GB GPUs and the code has not been released. We could not reproduce the full system on a laptop. What we did was reimplement its mask equation (their Eq. 8) at the same size as our head, on our features, and compare the two directly.

TABLE III
RECENT METHODS ON THE AXES THAT MATTER HERE.

Method	Gives region	Region scored	Trainable
AdaptEdit [1]	yes	no	5,033 M
WhereEdit [3]	internal	no	–
MaskFlow [4]	needs one	–	–
GPT-4o / Gemini [14]	hidden	no	closed
CLIPSeg [25]	yes	as segmentation	150.7 M
G-DINO + SAM 2.1 [20], [21]	yes	not for edits	0 (frozen)
Ours v2	**yes**	**IoU, 503 edits**	**4.69 M**

D. Comparison with recent methods (SOTA)

III. METHODOLOGY

A. System overview

The full pipeline is drawn in Fig. 1. We only trained the edit-region head. All the other models are used as

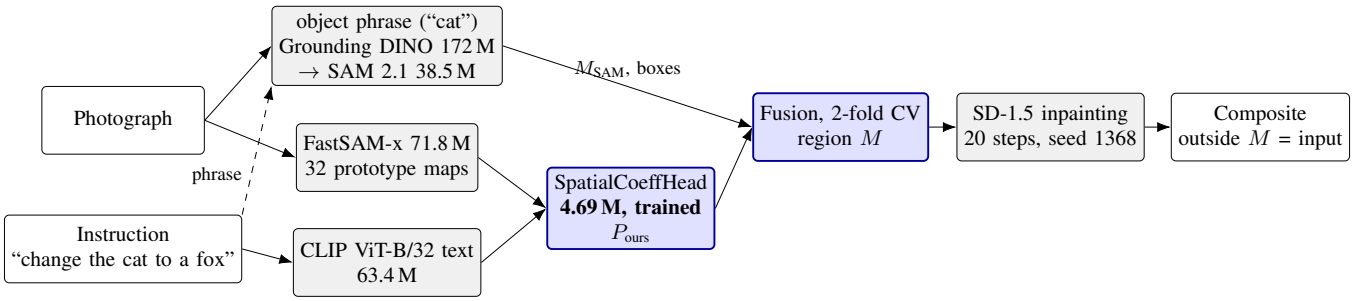


Fig. 1. Block diagram. Grey blocks are pretrained and frozen; blue blocks are ours. The region is decided before any pixel is painted, and the composite step copies every pixel outside it from the input.

TABLE IV
WHAT IS TRAINED AND WHAT IS REUSED.

Component	Params	Status
FastSAM-x [22] (YOLOv8x-seg)	71.8 M	frozen, published weights
CLIP ViT-B/32 text [26]	63.4 M	frozen
Grounding DINO tiny [21]	172.3 M	frozen
SAM 2.1 Hiera-S [20]	38.5 M	frozen
SD-1.5 inpainting [27]	∼0.9 B	frozen editor
SpatialCoeffHead (ours)	**4.69 M**	**trained from scratch**
Fusion (d, a, t)	3 values	chosen by 2-fold CV

released, without any fine-tuning (Table IV).

B. Edit-region head

For any photo, FastSAM [22] gives 32 prototype maps P_i of size 160×160 and a pooled 640-d context vector. Since FastSAM does not look at the instruction, this only has to run once per image. CLIP turns the instruction into a 512-d vector $\mathbf{t}$. Our head combines the two using FiLM and outputs a 20×20 grid of coefficients, which is upsampled to 160×160:

$$P_{\text{ours}}(x, y) = \sigma\Big(\sum_{i=1}^{44} c_i(\mathbf{t}, x, y)\, B_i(x, y) + b \Big) \qquad (1)$$

Here B_i are the 32 prototypes and 12 fixed geometric maps. So the head does not have to produce a mask shape from scratch. What it learns is a set of weights that say how much of each map to use at each location, and we think this is a large part of why such a small head works at all.

The base paper's MaskPredictor works in a different way. A grid of learned queries is modulated by the instruction, attends over the image patches, and is then decoded into a mask. We rebuilt it as `QueryGridHead` with 4,673,777 parameters and trained it with exactly the same script, data and seeds as ours.

C. Supervision and training

For each MagicBrush [23] training turn, we took the source and edited photos, computed the CIE76 ΔE colour difference in L*a*b* space and thresholded it at 12. We then cleaned the result with a 5×5 morphological filter, removed blobs smaller than 40 pixels, and kept only turns where the changed area was between 0.4% and 45% of the image. Fig. 2 shows each step on a real example. That left 8,306 turns for training, with the settings listed in Table VI.

TABLE V
DRAWN MASKS AGAINST THE COMPUTED TARGET, MAGICBRUSH TRAIN.

Region	Frame covered	Precision
MagicBrush drawn mask	62.4%	15.8%
ΔE **target (ours)**	**10.3%**	reference

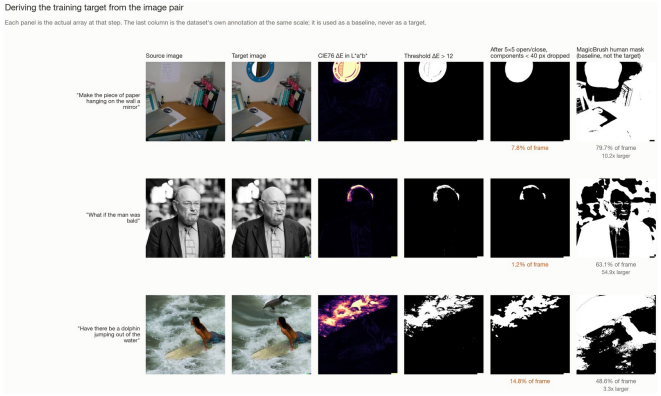


Fig. 2. Computing the target from a before/after pair. Each panel is the real array at that step.

TABLE VI
TRAINING SETUP FOR THE HEAD.

Loss	BCE + 2× Dice
Optimizer	AdamW, lr 10^{-3}, weight decay 0.01
Schedule	cosine, 60 epochs, batch 32, grad clip 1.0
Checkpoint choice	15% slice of train, never the test split
Seeds	12 (1368, 1–11)
Input	640 px; prototypes at 160 px

We did not train on the hand-drawn masks, and the reason is simple. As Table V shows, they are about six times larger than the area that actually changes, and a model trained on them would learn to predict regions that are far too big.

D. Ours v2: building on the strongest localizer

First, a simple rule picks out the object named in the instruction (for “change the cat to a fox” it gives “cat”).

TABLE VII
DATA USED.

Dataset	Split	Turns	Use
MagicBrush [23]	train	8,306	train the head
MagicBrush	dev (held out)	503	all IoU figures
Emu Edit [24]	test	56	transfer only

Grounding DINO [21] finds boxes for that phrase and SAM 2.1 [20] cuts out the object inside each box, giving M_{SAM}. SAM's outlines are tight around the object, while our head has learned that edits also spill into shadows and nearby pixels. We combine the two as

$$M = \left[\, a \,\mathrm{dil}_d(M_{\text{SAM}}) + (1-a)\, P_{\text{ours}} \mathbb{1}_{\text{boxes}} > t \,\right] \quad (2)$$

We chose d, a and t by two-fold cross-validation. We tuned on one half of the 503 turns and scored on the other half, and then swapped the halves, so every reported score comes from data that was not used for tuning. Both folds settled on $d = 2$ px and $a = 0.5$. One fold preferred $t = 0.5$ without the box gate and the other $t = 0.3$ with it. The website uses the second setting.

E. Editing and compositing

To edit, we turn the instruction into a short description ("a fox"), grow the region by 8 px and let SD-1.5 inpainting [27] fill it in (20 steps, guidance 7.5, seed 1368). The result is then pasted onto the original photo through the region mask, so every pixel outside it comes straight from the input.

Colour changes such as "make the bed red" needed a separate path. An inpainter repaints the region from nothing. In our first tests, asking it to make a dog black produced a completely different black dog. For these instructions we change the colour in L*a*b* space instead, taking the chroma of the target colour and keeping the original lightness, so the object keeps its shape and shading. Both paths are applied in the same way for all three predictors, and any other inpainting model could replace SD-1.5 without changing the localization part.

IV. Results and Discussion

A. Data, setup and metric

Data. Of the 503 held-out turns, 318 modify something, 145 insert something and 40 remove something. We checked for leakage and no image appears in both the training and the held-out sets. Emu Edit does not include edited target images, so there is no true region to score against; we use it only to see how well the model transfers.

Setup. Everything, training included, was done on a single Apple M5 laptop (10 cores, 26 GB, MPS backend) with PyTorch 2.14 and Python 3.12. At 352 px the frozen backbone needs 33.6 ms per image, and our head adds 5.3 ms per instruction. CLIPSeg needs 54.3 ms for the same job. On the website, Grounding DINO, SAM 2.1 and our head produce a region in 1.1 to 5.1 s, and the edit itself takes another 15 to 40 s.

TABLE VIII
IoU ON 503 HELD-OUT TURNS (12 SEEDS FOR TRAINED HEADS).

Region source	Trainable	IoU
Random centred box	0	0.0621
Whole frame	0	0.0958
AdaptEdit Eq. 8 head (reimpl.)	4.67 M	0.1314
MagicBrush human mask	hand-drawn	0.1511
CLIPSeg [25]	150.7 M	0.1849
Ours (head, mean of 12)	4.69 M	0.2065
G-DINO + SAM 2.1	0 (frozen)	0.2721
Ours v2 (2-fold CV)	**4.69 M**	**0.2771**

TABLE IX
MODEL SELECTION BY EDIT KIND (IoU, 503 TURNS). THE HEAD ROW IS THE SINGLE RUN (SEED 1368) USED IN THE SYSTEM; TABLE VIII GIVES THE 12-RUN MEAN, 0.2065.

Localizer	All	Insert	Modify	Remove
CLIPSeg	0.1849	0.0936	0.2121	0.2993
Ours (head, seed 1368)	0.2055	0.1384	0.2369	0.1987
G-DINO + SAM 2.1	0.2721	0.1445	0.3108	0.4270
Ours v2	**0.2771**	**0.1487**	**0.3170**	0.4259

Metric. We report IoU between each predicted region and the ΔE region, which is the set of pixels that changed when a person made the edit. This reference comes straight from the data and not from any of the models, so every row in the tables below is measured against the same thing.

B. Localization accuracy

Our head scores higher than CLIPSeg while training 32 times fewer parameters. All twelve training seeds ended up above CLIPSeg ($p = 5.2\times10^{-7}$, with a spread of 0.0072 across seeds), and it does better on 307 of the 503 photos. The IoU values look low. That is mostly because the target is a thresholded pixel difference, which is noisy around object edges.

Of the open models we tried, Grounding DINO with SAM 2.1 was the best, and it was also much better than our head on removals, which had been our weak spot. Adding our head on top (ours v2) helped on 273 photos and hurt on 173 (paired $t = 2.16$). Most of the gain is on insertions and modifications, where the edit tends to spread beyond the object that was named.

C. Effect on the edited photograph

When the editor was given our region (Fig. 3), it changed 12.6% of the pixels that should have stayed the same. Without a region that figure was 93.1%,

TABLE X

SAME EDITOR AND NOISE; ONLY THE REGION DIFFERS (40 TURNS). NET = RECALL − COLLATERAL.

Region given to editor	Recall	Collateral	Net
Whole frame	94.9%	93.1%	+1.8%
MagicBrush human mask	77.4%	45.7%	+31.6%
Ours	50.0%	**12.6%**	**+37.4%**
Ground truth (ceiling)	57.2%	0.0%	+57.2%

TABLE XI

THE BASE PAPER'S METRIC SUITE ON OUR PIPELINE (60 TURNS, SAME EDITOR).

Region	L1 ↓	L2 ↓	CLIP-I ↑	DINO ↑
Whole frame	0.2763	0.1180	0.6242	0.2693
Human mask	0.1662	0.0660	0.7919	0.6343
Ours	**0.0713**	**0.0279**	**0.8879**	**0.7986**
Ground truth	0.0318	0.0110	0.9382	0.9103

TABLE XII

EMU EDIT TEST, NEVER USED IN TRAINING (56 SAMPLES, 8 EDIT KINDS).

Region	CLIP-I to input	L1 to input	Area
Whole frame	0.5017	0.2915	100%
CLIPSeg	0.7111	0.1664	55.5%
Ours	**0.8519**	**0.0813**	39.5%

and PSNR outside the region went up from 9.92 to 21.17 dB. We also used the metrics from the base paper on 60 turns. With our region we got L1 0.0713, CLIP-I 0.8879 and DINO 0.7986, compared with 0.1662, 0.7919 and 0.6343 using the human masks. On twelve photos where we compared against the annotator's own edited image, ours was closer in L1 for 11 of them (0.0577 against 0.0915). On Emu Edit, which the model never saw during training, CLIP-I to the input stayed at 0.8519 for ours and 0.7111 for CLIPSeg.

D. Ablation study

Same size, different mechanism. We trained both heads at four matching sizes, three seeds each (Fig. 5). Mixing a fixed set of maps beat decoding from a query grid by 0.062 IoU at the smallest size, and the gap grew to 0.135 at the largest. This suggests the gap comes from how the mask is built rather than from model size, although we have only checked it on FastSAM features. We also tried giving the query grid the same resolution as ours, but that only changed its score by 0.0035.

Threshold. As Table XIII shows, the net gain goes from +29.7% at a threshold of 0.5 up to +38.2% at 0.2, and changes very little below that, so we picked 0.2. With only 40 turns per row, we would not read much into the small differences between 0.1 and 0.2.

Free-space basis. We tried adding extra maps to help with insertions. They did no better than the same

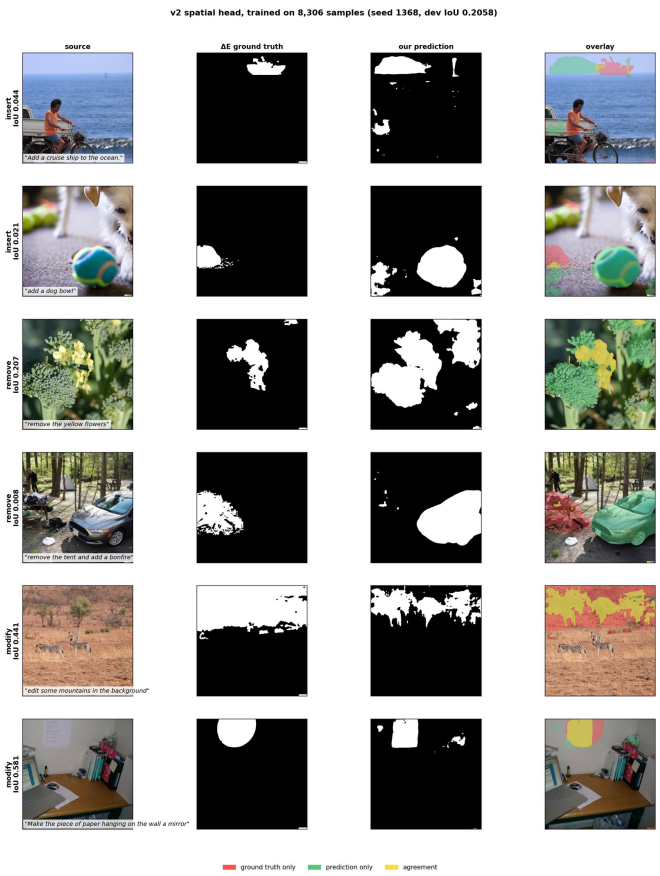


Fig. 3. Held-out examples by edit kind. Same editor and noise; only the region changes. Without a region (first column of results) the whole photo is redrawn.



Fig. 4. Held-out cases run live. Columns: base paper mechanism, G-DINO + SAM 2.1, ours v2. Same editor, prompt and seed.

number of random maps ($p = 0.33$), so we did not keep them. A better-designed set might still help; we just did not find one.

Fusion. With the same cross-validation, SAM alone

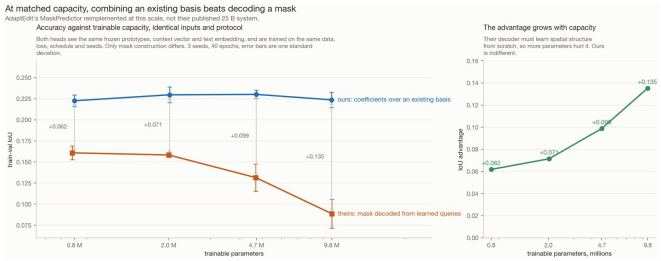


Fig. 5. Mixing a fixed basis (ours) against decoding a query grid (base paper Eq. 8) at four matched sizes, three seeds each.

TABLE XIII
THRESHOLD SWEEP AT 8 PX DILATION (40 TURNS PER ROW).

Threshold	Recall	Collateral	Net	Area
0.10	57.5%	19.8%	+37.7%	35.8%
0.15	53.2%	15.7%	+37.5%	30.8%
0.20	51.6%	13.4%	**+38.2%**	27.2%
0.30	45.0%	9.7%	+35.3%	21.8%
0.40	39.5%	7.4%	+32.1%	17.5%
0.50	35.3%	5.6%	+29.7%	14.1%

scores 0.2722 and our head alone 0.2048, while the combination reaches 0.2771.

E. Failure analysis

On its own, our head seems to struggle most with removals (0.1987, against 0.2993 for CLIPSeg). Our best explanation is that when an object is removed, the region also has to include what becomes visible behind it, and none of the prototype maps for that object covers that area. Ours v2 gets around this through SAM and scores 0.4259 on removals. Insertions were hard for every model we tried, and none scored above 0.149. We expected some of this, since there is nothing in the photo to detect yet, but we have not looked closely at which insertions fail and why. Finally, the picture quality is set by SD-1.5. Even with the perfect region, its L1 only improves from 0.0713 to 0.0318.

F. Comparison with commercial editors

A question we were asked during reviews was why not simply use ChatGPT or Gemini. Their images look better than what SD-1.5 gives us, and we do not claim otherwise. The difference is in control. Going by the GPT-ImgEval report [14], GPT-4o regenerates the whole image even when only a small part is meant to change, and both it and Gemini drift as edits are repeated. We could not run those models on our benchmark, so Table XIV only lists what we could check on our side.

G. Deployment

The system is live at https://localize-edit.onrender.com (Fig. 6). The front end is a static site on Render. It has ten example photos, an upload button, an instruction box you can edit, the three predictors side by side, a before/after slider, and follow-up prompts. Render does not offer GPUs, so the models run in a FastAPI server on our M5 laptop, which the site reaches through a Cloudflare tunnel. The site reads the server's current address from a small file in the repository, so the server could be moved to any other GPU machine without rebuilding the site.

Ours v2 had the highest IoU in all ten cases (Fig. 4 shows five of them). We should point out that we picked these ten from held-out turns where our model gains a lot, so they show the effect clearly but are not a fair sample. The fair comparison is Table VIII, where ours v2 is ahead on 273 of the 503 turns and behind on 173. Cases 2 and 3 are colour changes and went through the

TABLE XIV
CLOSED EDITORS VS. OUR LOCALIZATION LAYER.

	GPT-4o / Gemini	Ours
Region shown before editing	no	yes
Pixels outside region	regenerated [14]	copied; 0.00% changed in 10/10 live runs
Multi-turn drift	reported [14]	21.1% after 2 turns
Localization measured	no	IoU 0.2771
Region time	n/a	1–5 s, laptop
Photo leaves the device	yes	no
Picture quality	high	editor-bound

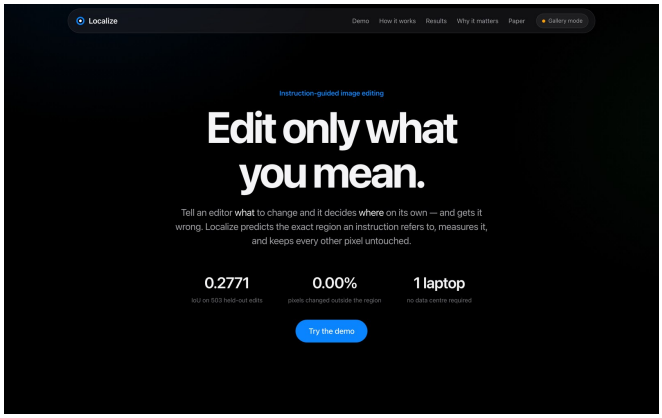


Fig. 6. The deployed application.

TABLE XV
THE TEN GALLERY CASES RUN LIVE THROUGH THE PUBLIC SITE (24 OCT 2026). EACH IS A MAGICBRUSH HELD-OUT TURN, SO THE SITE SCORES EVERY REGION AGAINST THE TRUE EDIT REGION (IOU). OUT: % OF PIXELS CHANGED OUTSIDE OUR REGION. A FULL RUN OF ALL THREE TOOK 89 S ON AVERAGE.

#	Instruction	Base	GSAM	Ours v2	Out
1	change the cat to a fox	0.295	0.636	**0.680**	0.00
2	The bed should be red	0.127	0.041	**0.422**	0.00
3	make the frisbee blue	0.016	0.860	**0.929**	0.00
4	It should be a chocolate cake on the plate	0.399	0.104	**0.720**	0.00
5	change the bowl of fruit into a plate of salad	0.050	0.645	**0.739**	0.00
6	let it be a single sink	0.356	0.618	**0.734**	0.00
7	Replace the fruit with more veggies	0.226	0.446	**0.626**	0.00
8	let the plate contain ice cream	0.031	0.095	**0.605**	0.00
9	edit some mountains in the background	0.427	0.435	**0.585**	0.00
10	let the man be dressed in a dinner jacket	0.413	0.591	**0.703**	0.00
	Mean	0.234	0.447	**0.674**	

TABLE XVI
SHARE OF THE PHOTOGRAPH CHANGED FROM THE ORIGINAL.

Predictor	Turn 1	Turn 2
Base paper mechanism	41.5%	73.6%
G-DINO + SAM 2.1	16.6%	17.1%
Ours v2	**17.4%**	**21.1%**

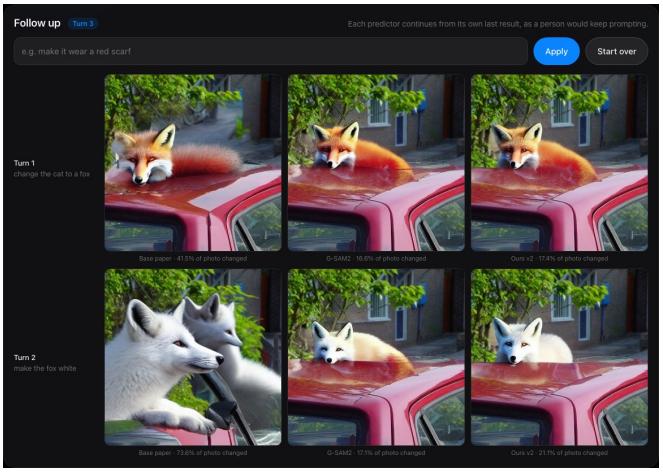


Fig. 7. "Change the cat to a fox", then "make the fox white". The base mechanism adds a second fox and rebuilds the car.

recolouring path.

H. Follow-up prompts

In a follow-up, each predictor keeps editing its own previous result (Fig. 7). We count a pixel as changed if any of its colour channels is more than 24/255 away from the original.

In practice, once one edit has damaged part of the photo, the next prompt works on top of the damaged version. The user then usually has to go back to the original and try again, which costs extra prompts. We also tried a third turn that recoloured the whole car, and it disturbed the fox for all three predictors, which we put down to the editor.

V. Conclusion and Future Work

Once localization is split off from editing, the region becomes something we can look at, measure and improve. Our 4.69 M head on frozen features scored higher than a 150.7 M referring segmenter and higher than the dataset's own masks. Combined with Grounding DINO and SAM 2.1, it reached an IoU of 0.2771 on 503 held-out edits. Using our region, the share of pixels the editor changed outside the target dropped from 93.1% to 12.6%, and the website keeps the rest of the scene unchanged over follow-up prompts.

There are a few things we would like to do next. The first is to put the same region in front of a stronger editor such as Qwen-Image-Edit or FLUX.1 Kontext, since picture quality is now limited by SD-1.5. We also want to predict the background that appears when an object is removed. The mechanism comparison was only run on FastSAM features, and repeating it on SAM and DINO features would tell us whether the result holds more generally. Last, a small user study could count how many prompts people need to reach the edit they want.

References

[1] H. Cai *et al.*, "Edit Where You Mean: Region-Aware Adapter Injection for Mask-Free Local Image Editing," NeurIPS, 2026. arXiv:2604.23763

[2] J. He *et al.*, "Rethinking Where to Edit: Task-Aware Localization for Instruction-Based Image Editing," 2026. arXiv:2604.20258

[3] M. Hu *et al.*, "WhereEdit: Mask-aware Local Latent Editing for One-Step Image Editing," 2026. arXiv:2607.20883

[4] R. Xu *et al.*, "MaskFlow: Precise, Consistent and Seamless Regional Image Editing," 2026. arXiv:2608.06929

[5] W. Chow *et al.*, "Masked Generative Transformer Is What You Need for Image Editing," 2026. arXiv:2605.10859

[6] S. Wu *et al.*, "LocateEdit-Bench: A Benchmark for Instruction-Based Editing Localization," 2026. arXiv:2602.05577

[7] X. Zang *et al.*, "Instruction-based Image Editing: A Survey on Data, Models, Evaluation, and Applications," 2026. arXiv:2607.25642

[8] P. Chen *et al.*, "RegionE: Adaptive Region-Aware Generation for Efficient Image Editing," 2025. arXiv:2510.25590

[9] Z. Qin *et al.*, "SpotEdit: Selective Region Editing in Diffusion Transformers," 2025. arXiv:2512.22323

[10] H. Wu *et al.*, "NEP: Autoregressive Image Editing via Next Editing Token Prediction," 2025. arXiv:2508.06044

[11] S. Liu *et al.*, "Step1X-Edit: A Practical Framework for General Image Editing," 2025. arXiv:2504.17761

[12] Black Forest Labs, "FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space," 2025. arXiv:2506.15742

[13] Qwen Team, "Qwen-Image Technical Report," 2025. arXiv:2508.02324

[14] Z. Yan *et al.*, "GPT-ImgEval: A Comprehensive Benchmark for Diagnosing GPT4o in Image Generation," 2025. arXiv:2504.02782

[15] Z. Li *et al.*, "Balancing Preservation and Modification: A Region and Semantic Aware Metric for Instruction-Based Image Editing," 2025. arXiv:2506.13827

[16] T. P. Vu *et al.*, "Region in Context: Text-condition Image Editing with Human-like Semantic Reasoning," 2025. arXiv:2510.16772

[17] T. Qu *et al.*, "RePlan: Reasoning-guided Region Planning for Complex Instruction-based Image Editing," ECCV, 2026. arXiv:2512.16864

[18] Z. Zhan *et al.*, "MADiff: Text-Guided Fashion Image Editing with Mask Prediction and Attention-Enhanced Diffusion," 2024. arXiv:2412.20062

[19] Y. Nitzan *et al.*, "Lazy Diffusion Transformer for Interactive Image Editing," ECCV, 2024. arXiv:2404.12382

[20] N. Ravi *et al.*, "SAM 2: Segment Anything in Images and Videos," 2024. arXiv:2408.00714

[21] S. Liu *et al.*, "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," ECCV, 2024. arXiv:2303.05499

[22] X. Zhao *et al.*, "Fast Segment Anything," 2023. arXiv:2306.12156

[23] K. Zhang *et al.*, "MagicBrush: A Manually Annotated Dataset for Instruction-Guided Image Editing," NeurIPS, 2023. arXiv:2306.10012

[24] S. Sheynin *et al.*, "Emu Edit: Precise Image Editing via Recognition and Generation Tasks," CVPR, 2024. arXiv:2311.10089

[25] T. Lüddecke and A. Ecker, "Image Segmentation Using Text and Image Prompts," CVPR, 2022. arXiv:2112.10003
[26] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021. arXiv:2103.00020
[27] R. Rombach *et al.*, "High-Resolution Image Synthesis with Latent Diffusion Models," CVPR, 2022. arXiv:2112.10752